**TASK 13: REST API with Exception Handling & Validation**

**AIM**

To enhance a Spring Boot RESTful application by adding input validation using validation annotations and implementing global exception handling to return meaningful error responses.

**ALGORITHM**

1. Start
2. Create Spring Boot project with Web + Validation dependency
3. Create Product entity with validation annotations
4. Create REST controller for CRUD operations
5. Create custom exception class
6. Create Global Exception Handler using @ControllerAdvice
7. Validate input using @Valid
8. Return meaningful error messages in JSON format
9. Test using Postman
10. End

**Code :**

```
<!DOCTYPE html>
<html>
<head>
    <title>Product API Output - Validation & Exceptions</title>
    <style>
        body {
            font-family: Arial;
            background: #f4f4f4;
            text-align: center;
        }
        .container {
```

```
    width: 70%;
    margin: auto;
    background: white;
    padding: 20px;
    margin-top: 30px;
    border-radius: 10px;
    box-shadow: 0px 0px 10px #ccc;        }
h2 {
    color: #4CAF50;
}
input {
    padding: 10px;
    margin: 5px;
    width: 200px;
button {
    padding: 10px 15px;
    margin: 5px;
    cursor: pointer;
}
.add { background: green; color: white; }
table {
    width: 90%;
    margin: auto;
    border-collapse: collapse;
    margin-top: 20px;
}
th, td {
    border: 1px solid #ccc;
    padding: 10px;
```

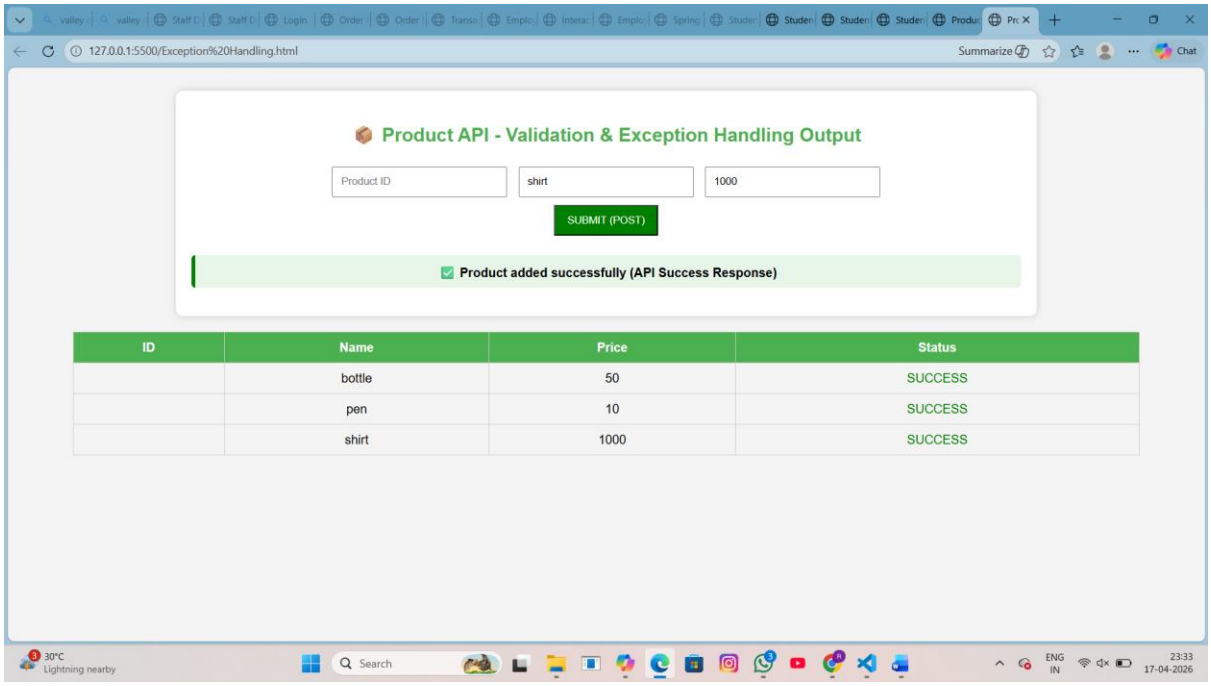
```
    text-align: center;
}
th {
    background: #4CAF50;
    color: white;
}
#msg {
    font-weight: bold;
    margin-top: 10px;
}
.success {
    color: green;
}
.error {
    color: red;
}
.box {
    margin-top: 20px;
    padding: 10px;
    border-radius: 5px;
}
.successBox {
    background: #e8f5e9;
    border-left: 5px solid green;
}
.errorBox {
    background: #ffebee;
    border-left: 5px solid red;
}
```

```
    </style>
</head>
<body>
<div class="container">
  <h2>📄 Product API - Validation & Exception Handling Output</h>
    <!-- INPUT FORM -->
    <input type="number" id="id" placeholder="Product ID">
    <input type="text" id="name" placeholder="Product Name">
    <input type="number" id="price" placeholder="Product Price">
    <br>
    <button class="add" onclick="simulateAPI()">SUBMIT (POST)</button>
    <p id="msg"></p>
</div>
<!-- OUTPUT TABLE -->
<table>
    <thead>
        <tr>
            <th>ID</th>
            <th>Name</th>
            <th>Price</th>
            <th>Status</th>
        </tr>
    </thead>
    <tbody id="tableData"></tbody>
</table>
<script>
// ==========================
// SIMULATED DATABASE
// ==========================
```

```
let products = [];
// ==========================
// DISPLAY FUNCTION
// ==========================
function display() {
    let table = "";
    products.forEach(p => {
        table += `
        <tr>
            <td>${p.id}</td>
            <td>${p.name}</td>
            <td>${p.price}</td>
            <td class="success">SUCCESS</td>
        </tr>`;
    });
    document.getElementById("tableData").innerHTML = table;
}
// ==========================
// VALIDATION + EXCEPTION SIMULATION
// ==========================
function simulateAPI() {
    let id = document.getElementById("id").value;
    let name = document.getElementById("name").value;
    let price = document.getElementById("price").value;
    let errors = [];
    // VALIDATION CHECKS
    if (name === "") {
        errors.push("Product name cannot be empty");
    }
```

```
    if (price <= 0) {
        errors.push("Price must be greater than 0");
    }
    // ERROR HANDLING (LIKE @ControllerAdvice)
    if (errors.length > 0) {
        document.getElementById("msg").innerHTML =
            `<div class="box errorBox">
                ❌ Validation Failed:<br>${errors.join("<br>")}
            </div>`;
        return;
    }
    // SUCCESS CASE (LIKE REST API RESPONSE)
    products.push({ id, name, price });
    document.getElementById("msg").innerHTML =
        `<div class="box successBox">
            ✅ Product added successfully (API Success Response)
        </div>`;
    display();
}
</script>
</body>
</html>
```

**Output :**




**RESULT:**

A Product API Output Page was successfully created using HTML, CSS, and JavaScript. The system simulates Spring Boot validation and global exception handling. It displays both success responses and error messages dynamically, demonstrating how REST APIs handle valid and invalid input in real-world applications.